
Contamination Contagion: How Benchmark Leakage in Pre-Training Propagates Through Fine-Tuning and Corrupts Downstream Evaluation

Anonymous Author(s)

Affiliation

Address

email

Abstract

Modern language models are routinely fine-tuned on top of open-weight base models whose pre-training corpora are rarely audited. If benchmark data was present during base-model pre-training, a practitioner who fine-tunes on unrelated downstream data may inherit inflated scores without ever seeing the contaminated inputs. We study this *contamination contagion* directly. We inject GSM8K, MMLU, and HumanEval test examples into clean SlimPajama pre-training at three controlled rates (0.1%, 1.0%, 5.0%), continue pre-training two families of open models, fine-tune the contaminated checkpoints on four unrelated downstream tasks, and measure what fraction of the benchmark inflation survives. We introduce the *Contamination Persistence Rate* (CPR), a paired metric that isolates how much contamination a fine-tuning run retains relative to the underlying base-contaminated gap, and the *Contamination Transfer Score* (CTS), which quantifies cross-benchmark inflation on related evaluations that were never directly contaminated. Across 108 training runs totaling 200 A40-hours, we report CPR, CTS, and detection AUROC for every (model, rate, task) cell with 95% paired-bootstrap confidence intervals over per-example correctness indicators and three random seeds. We additionally compare four contamination detectors (Min-K% Prob, perplexity ratio, verbatim completion, and a mid-layer representation-space probe) at the pre-fine-tune and post-fine-tune stages, reporting AUROC against held-out reference examples. Our pipeline, contamination manifests with SHA-256 hashes, and per-example predictions are released so that the community can audit existing fine-tuned model releases for inherited base-model contamination.

1 Introduction

Public benchmarks are the principal yardstick by which progress in large language models is communicated and compared. Their credibility rests on the assumption that test examples have not been seen during training. That assumption has been repeatedly shown to fail in practice: recent audits have uncovered measurable leakage of MATH, AIME, MMLU, and HumanEval examples into the pre-training corpora of widely used open and closed models [Zhou et al., 2024, Yang et al., 2024, Wu et al., 2025a]. A parallel line of work has shown that most forms of memorization can in principle be detected while the base model is accessible for inference, provided the right probing technique is chosen [Shi et al., 2024, Zhang et al., 2025a, Xie et al., 2024].

What is almost never studied is what happens *after* a base model is fine-tuned. Modern deployment pipelines rarely use the raw base checkpoint. A practitioner who wants a customer-support assistant, a code reviewer, or a medical triage summarizer will LoRA-tune or supervised-fine-tune a released base model on their in-domain data, then publish the adapter or merged weights. The downstream

model inherits the base model’s representations in ways that the fine-tuning data itself does not control. If the base model was contaminated during pre-training, does the fine-tuned derivative remain contaminated? If so, how strongly, and can the standard contamination detectors still surface it?

We answer these questions empirically. We design a controlled contamination pipeline that mirrors the most realistic failure mode: test-set examples reformatted as natural pre-training text and mixed into a clean corpus at token-level rates of 0.1%, 1.0%, and 5.0%. We continue pre-training two open model families, Qwen3-8B and Llama-3.1-8B, on the resulting shards, then perform supervised fine-tuning of each contaminated checkpoint on four downstream tasks that share no surface content with the contaminated benchmarks (NER on CoNLL-2003, extractive QA on SQuAD 2.0, abstractive summarization on CNN/DailyMail, code synthesis on MBPP). We evaluate every stage of the pipeline on seven benchmarks and quantify the surviving inflation using a paired Contamination Persistence Rate, a Contamination Transfer Score for cross-benchmark leakage, and four detection methods applied at both the pre- and post-fine-tune stages. Our experiments cover supervised fine-tuning only; we do not test RLHF or DPO post-training and we discuss this scope limitation in Section 6.

Contributions. Our main contributions are:

- The first systematic study of contamination propagation through the pre-train → fine-tune pipeline, covering two model families, three benchmarks, three contamination rates, and four downstream tasks with paired seeds.
- Two reusable metrics, CPR(persistence through fine-tuning) and CTS(cross-benchmark transfer), together with an open implementation and per-example bootstrap confidence intervals for both.
- An evaluation of four contamination detectors (Min-K% Prob [Shi et al., 2024], perplexity ratio, verbatim completion, and a mid-layer representation-space probe) at both the pre-fine-tune and post-fine-tune stages, with AUROC against held-out reference examples reported for every cell.
- A reproducible audit script that, given a released base model and its fine-tuned derivative, returns a paired CPReestimate and per-detector AUROC; together with practical guidance on when a fine-tuned benchmark score should be reported as a point estimate vs. as an interval that disclosed the inherited contamination uncertainty.

2 Related Work

Our work sits at the intersection of benchmark contamination detection, contamination-resistant evaluation design, fine-tuning dynamics, and the emerging crisis of trust in LLM benchmarks. We organize prior work into five themes and identify the gap that motivates our study.

2.1 Benchmark Contamination Detection

The problem of detecting whether evaluation data has leaked into training corpora has received substantial attention. **Probability-based methods** pioneered by Shi et al. [2024] introduced Min-K% Prob, which flags contamination by examining the average log-probability of the $k\%$ least probable tokens in a sequence, under the hypothesis that unseen examples contain more low-probability outlier tokens. This was subsequently improved by Zhang et al. [2025a], who provided theoretical grounding by framing detection as identification of local maxima in the modeled distribution. Xie et al. [2024] proposed ReCaLL, a membership inference attack leveraging the insight that conditioning on non-member prefixes induces larger log-likelihood drops for member data. More recently, Choi et al. [2025] introduced the Kernel Divergence Score (KDS), which measures changes in kernel similarity matrices before and after fine-tuning on benchmark data, achieving near-perfect correlation with contamination levels.

Prompt-based detection approaches include Golchin and Surdeanu [2024]’s “guided instruction” method, where models are prompted with dataset names and partial instances to test completion ability, achieving 92–100% accuracy. Deng et al. [2024a] introduced Testset Slot Guessing, revealing that GPT-4 can guess 57% of missing MMLU options. Yang et al. [2024] demonstrated that simple n-gram overlap methods miss rephrased contamination, proposing an LLM-based decontaminator that found 8–18% HumanEval overlap in common pre-training corpora.

86 **N-gram and perplexity-based methods** remain widely used despite known limitations. Zhou et al.
 87 [2024] developed a pipeline combining perplexity and n-gram accuracy to analyze 31 LLMs, revealing
 88 substantial test set misuse in mathematical reasoning and proposing the Benchmark Transparency
 89 Card. Anonymous [2025g] compared detection methods under realistic continual pre-training,
 90 finding n-gram methods achieve the highest F1 scores. In the emerging area of post-training detection,
 91 Anonymous [2025f] proposed Self-Critique, the first method targeting RL post-training contamination,
 92 achieving up to 30% AUC improvement by probing for policy collapse.

93 A critical finding across this literature is that **no detection method operates across the full training**
 94 **pipeline**. Each targets a single stage—pre-training [Shi et al., 2024, Zhang et al., 2025a], supervised
 95 fine-tuning [Anonymous, 2024d], or RL post-training [Anonymous, 2025f]—without tracking how
 96 contamination signals transform across stages. Our work fills this gap by measuring detection method
 97 efficacy at each pipeline stage for the same underlying contamination.

98 2.2 Contamination-Resistant Benchmarks

99 The complementary approach to detecting contamination is preventing it through benchmark design.
 100 White et al. [2025] introduced LiveBench, which draws from recent math competitions, arXiv papers,
 101 and news articles with monthly updates, establishing the gold standard for temporal contamination
 102 resistance. Jain et al. [2024] applied similar principles to code evaluation, continuously collecting
 103 problems from competitive programming platforms. Domain-specific variants include LiveMedBench
 104 [Anonymous, 2026] for clinical medicine, which revealed that 84% of models degrade on post-cutoff
 105 cases.

106 An alternative strategy generates test instances algorithmically. Anonymous [2025b] introduced
 107 BeyondBench with over 10^{15} unique instances across 44 algorithmic tasks with deterministic verifica-
 108 tion, evaluating 101 models. Anonymous [2025e] uses multi-agent systems to generate semantically
 109 equivalent code problem variations, revealing significant performance degradation suggestive of
 110 contamination.

111 Zhang et al. [2024] created GSM1K, a human-authored mirror of GSM8K, finding up to 8% accuracy
 112 drops and systematic overfitting in Mistral and Phi model families—a landmark result published
 113 at the NeurIPS 2024 Datasets and Benchmarks Track. Wang et al. [2024] developed MMLU-Pro
 114 with 10-option multiple choice and harder questions, producing 16–33% accuracy drops relative to
 115 MMLU. Anonymous [2025a] constructed AntiLeak-Bench using Wikidata/Wikipedia updates to
 116 guarantee contamination-free evaluation. In the software engineering domain, Zhang et al. [2025b]
 117 audited 83 benchmarks, finding generally low leakage (4.8% average for Python) but extreme cases
 118 such as QuixBugs at 100%.

119 While these benchmarks address contamination *for future evaluation*, they cannot retroactively
 120 assess whether *existing fine-tuned models* carry contamination inherited from their base models—the
 121 question we address.

122 2.3 Contamination Propagation and Forgetting

123 The most directly related line of work studies how contamination behaves under continued training.
 124 Bordt et al. [2025] systematically studied forgetting along three scaling dimensions (parameters up to
 125 1.6B, repetitions up to $144\times$, training tokens up to 40B), finding that contamination can be “forgotten”
 126 when training data scales beyond $5\times$ Chinchilla—a regime characteristic of modern LLMs. However,
 127 their study examines only the pre-training phase and does not study what happens when fine-tuning
 128 is applied on top of potentially contaminated knowledge.

129 Zhang et al. [2026] is the closest existing work to ours, studying contamination propagation through
 130 LoRA adaptation in LLM-based recommendation systems. They identified a “Triple Effect” where
 131 domain-relevant leakage creates spurious performance gains while domain-irrelevant leakage degrades
 132 accuracy. Their study is limited to the recommendation domain and a single LoRA configuration
 133 on a single model family; our study differs by covering two independent model families, four
 134 distinct downstream task types, three contamination rates, and strict exact-match scoring on seven
 135 general-purpose benchmarks.

136 Two recent papers study the interaction between reinforcement learning and memorization. Wu
 137 et al. [2025a] demonstrated that Qwen2.5’s apparent mathematical reasoning gains from random or

incorrect rewards are artifacts of pre-training contamination on MATH-500 and AIME benchmarks, proposing the uncontaminated RandomCalculation alternative. Yan et al. [2026] provided mechanistic evidence via Path Patching and Logit Lens, identifying an “Anchor-Adapter circuit” (layers 18–20 anchoring memorized solutions, layers 21+ adapting representations) that enables memorization shortcuts during RLVR training. These findings are compelling but limited to the Qwen model family and mathematical reasoning tasks.

Our work differs from all of the above in three key respects. First, we study contamination propagation through pre-training followed by clean supervised fine-tuning on four *unrelated* downstream tasks, whereas Bordt et al. [2025] examine forgetting only during continued pre-training and Zhang et al. [2026] examine only in-domain LoRA fine-tuning. Second, we evaluate on seven general-purpose benchmarks (GSM8K, MMLU, HumanEval, MATH, MBPP, ARC-Challenge, TruthfulQA-MC) across two independent model families, rather than a single benchmark family or domain. Third, we measure whether four existing contamination detection methods remain effective at identifying contamination that has been transformed by downstream supervised fine-tuning, which no prior study has directly quantified. We do not evaluate RLHF or DPO post-training in this paper; that scope boundary is flagged explicitly as a limitation in Section 6.

2.4 Decontamination Methods

Several approaches attempt to mitigate contamination effects. Zhu et al. [2024] proposed Inference-Time Decontamination (ITD), which detects and rewrites leaked samples at evaluation time, reducing inflated accuracy by 22.9% on GSM8K and 19.0% on MMLU. Chai et al. [2026] extended this idea with DeconIEP, applying bounded perturbations in embedding space guided by less-contaminated reference models.

However, Sun et al. [2025] provided a sobering assessment: testing 20 mitigation strategies across 10 LLMs and 5 benchmarks, they found that **no existing strategy effectively balances fidelity and contamination resistance**. Semantic-preserving strategies offered no improvement over doing nothing, while semantic-altering strategies sacrificed benchmark validity. Xu et al. [2025] proposed a more nuanced approach with the DCR framework, quantifying contamination at four granular levels (semantic, informational, data, label) and producing adjusted accuracy scores within 4% of uncontaminated baselines. For code-specific applications, Anonymous [2025c] developed automated refactoring operators achieving 75% overlap reduction.

Our work reveals a further complication for decontamination: even if base model contamination is mitigated, fine-tuning may reactivate or transform residual contamination signals, necessitating pipeline-aware decontamination strategies.

2.5 Fine-Tuning Dynamics and Knowledge Retention

Understanding how fine-tuning affects pre-trained knowledge is essential background for our work. Wu et al. [2025b] showed that training on LLM-generated data (which exhibits lower perplexity) reduces non-target task degradation, proposing Selective Token Masking to achieve similar benefits with ground-truth data. Anonymous [2025i] found that upweighting samples with low pre-trained model loss preserves knowledge during SFT. Anonymous [2024b] documented that catastrophic forgetting intensifies with scale (1B–7B), and notably that LoRA does *not* mitigate forgetting in continual learning—contradicting common assumptions.

At the intersection of forgetting and safety, Anonymous [2024e] showed catastrophic forgetting can be leveraged positively for safety alignment, while Anonymous [2024c] established a power-law relationship between training steps and factual knowledge forgetting during pre-training.

These works study forgetting of *general knowledge*, but none specifically examine whether *memorized benchmark answers* are forgotten, retained, or amplified during supervised fine-tuning—the precise question our paper answers for the SFT stage.

2.6 Benchmark Trust Crisis

Our work is further motivated by the growing crisis of trust in LLM evaluation. The Llama 4 controversy, where Meta submitted a customized model variant to LMArena that ranked 2nd while

the public release ranked 32nd—later confirmed by Yann LeCun as “fudged” results—exemplifies the institutional pressures driving benchmark manipulation. Li et al. [2025a] identified “preference leakage” in LLM-as-a-judge paradigms, where judges systematically favor outputs from related models, compounding contamination concerns. Eriksson et al. [2025] conducted a meta-review of approximately 100 studies on benchmark shortcomings, cataloging issues from dataset biases to contamination to signal-noise confusion, concluding that the benchmark ecosystem faces a systemic validity crisis. Multiple surveys [Deng et al., 2024b, Xu et al., 2024, Li et al., 2025b, Anonymous, 2025d,h] have documented the growing scale of the contamination problem, with Anonymous [2024a] finding that 3 of 8 common detection assumptions fail across training phases.

Our paper contributes to resolving this crisis by demonstrating a previously uncharacterized failure mode: contamination in open-weight base models silently propagates to thousands of downstream fine-tuned variants, creating a systemic contamination problem that no amount of careful benchmark construction can fully address without awareness of the propagation mechanism.

3 Method

3.1 Threat model and design decisions

We simulate a realistic contamination event rather than a worst-case one. Benchmark test examples are reformatted into natural training text (Appendix B) so that exact-string deduplication pipelines are unlikely to catch them, but an interested reader can still verify the content. Contamination is token-weighted, not example-weighted, so that the same rate produces comparable memorization pressure across benchmarks of very different native size. We verify that the clean corpus shares no 13-gram with any contaminated test example before starting continued pre-training.

3.2 Continued pre-training with controlled injection

For each model and benchmark, we draw a fixed 50M-token shard from SlimPajama [Soboleva et al., 2023] and replace a specified token fraction with benchmark test examples. Continued pre-training uses AdamW ($\beta_1=0.9$, $\beta_2=0.95$, weight decay 0.1) at a conservative learning rate of $2e-5$, a cosine schedule with 5% warm-up, bfloat16 mixed precision, and gradient checkpointing. The effective batch size is 32 (per-device batch $2 \times$ gradient accumulation 16) at a sequence length of 2048 tokens, yielding 763 optimizer steps for the 50M-token budget. Because full-parameter updates of an 8B model exceed the 48 GB of a single A40 once AdamW state is allocated, we use LoRA rank 64 applied to all linear projections (q, k, v, o , gate, up, down). For an 8B model this leaves approximately 0.13% of parameters trainable, which is larger than the 2.5M contaminated tokens we inject at the highest rate (5%), so the adapter has strictly sufficient capacity to memorize every contaminated example; Appendix C makes this counting argument explicit and empirically compares verbatim-completion accuracy against a rank-128 and a full-FT NER ablation.

3.3 Clean downstream fine-tuning

Each contaminated checkpoint is used as the *starting point* for supervised fine-tuning on one of four downstream tasks. To make downstream training operate on the contaminated weights rather than an LoRA stack, we first merge the CPT adapter into the dense layers, then attach a fresh rank-16 LoRA adapter (alpha = 32, dropout 0.05, same target modules as CPT) and fine-tune with AdamW at learning rate $2e-4$, cosine schedule, 3% warm-up ratio, effective batch size 16 (per-device $2 \times$ grad-accum 8). Each task uses its natural epoch count: NER 3 epochs, QA 2 epochs, summarization 1 epoch, code 5 epochs. To stay within the compute budget across the full $2 \times 3 \times 4 \times 3 = 72$ FT cells, the QA and summarization training sets are uniformly subsampled to 10K and 5K examples respectively, NER uses the first 5K examples of CoNLL-2003, and code uses the full MBPP sanitized train split. The exact subset sizes are committed in `train_pipeline.py` and recorded in the pre-registered scope at `experiments/neurips-2026-002/SCOPE.md`. Fine-tuning data contains no overlap with any contaminated benchmark; we verify this explicitly with a 13-gram overlap check reported in Appendix E.

236 3.4 Metrics

237 Let A_b denote the accuracy of the unmodified base model on a benchmark, A_c the accuracy of the
 238 contaminated checkpoint prior to fine-tuning, A_f the accuracy of the base model fine-tuned on the
 239 same downstream task from a clean base, and A_{cf} the accuracy of the contaminated checkpoint
 240 fine-tuned on that task. We define

$$\text{CPR} = \frac{A_{cf} - A_f}{A_c - A_b}, \quad \text{CTS}_{X \rightarrow Y} = \frac{A_{cf}^Y - A_f^Y}{A_{cf}^X - A_f^X},$$

241 where X is the contaminated benchmark and Y is a related benchmark the model was never directly
 242 contaminated on. CPR is exactly zero when fine-tuning fully erases contamination and exactly one
 243 when contamination passes through unchanged; values above one indicate that the interaction between
 244 contamination and task-specific training amplifies leakage. CTS measures cross-benchmark inflation
 245 and equals zero for fully independent benchmarks.

246 3.5 Detection methods

247 We test four methods on every (model, rate, task) cell: Min-K% Prob [Shi et al., 2024], perplexity ratio
 248 against a clean CPT reference model, a linear SVM probe trained on the delta between contaminated
 249 and reference mid-layer embeddings, and verbatim completion of the second half of each benchmark
 250 example from a prefix prompt. All methods produce an AUROC over contaminated versus held-out
 251 reference examples.

252 4 Experimental Setup

253 4.1 Models

254 Qwen3-8B is our primary model family (public, non-gated, 8B parameters, grouped-query attention);
 255 Llama-3.1-8B serves as an architecture control with multi-head attention and a different pre-training
 256 corpus.

257 4.2 Benchmarks

258 GSM8K, MMLU, and HumanEval provide the contaminated benchmarks; MATH, ARC-Challenge,
 259 MBPP, and TruthfulQA-MC serve as transfer targets. All scoring uses strict exact-match with explicit
 260 answer-extraction regex to avoid generous string-containment false positives that inflate apparent
 261 post-FT accuracy [Wang et al., 2024]. Per-benchmark prompt templates and few-shot counts follow
 262 the standard evaluation conventions used in the HELM and Open LLM Leaderboard pipelines; exact
 263 templates are in Appendix A.

264 4.3 Seeds and reporting

265 Every cell is run with three seeds {42, 137, 2024}, which determine the CPT data shuffle and the
 266 Adam RNG at both training stages; the benchmark test splits are deterministic. For each cell we
 267 report the across-seed mean and standard deviation of per-example accuracy in addition to the point
 268 estimate. Confidence intervals are the 2.5%/97.5% quantiles of 10,000 paired bootstrap resamples
 269 over per-example correctness indicators. Pairwise significance tests use a paired bootstrap between
 270 clean-base-fine-tuned and contaminated-fine-tuned predictions on the same evaluation example.
 271 Across the full matrix (2 models $\times$ 3 contamination rates $\times$ 4 downstream tasks $\times$ 3 seeds = 72
 272 paired tests for the primary GSM8K experiments, plus 24 secondary-benchmark tests), p -values
 273 are adjusted using Benjamini–Hochberg FDR at $\alpha = 0.05$; we explicitly report which comparisons
 274 survive the correction. Effect sizes use pooled-variance Cohen’s d .

275 4.4 Compute

276 All experiments ran on single-A40 (48 GB) pods on a commodity cloud at an advertised rate of \$0.39
 277 per GPU-hour. The experimental matrix consists of 108 training runs (16 continued-pre-training
 278 injections $\times$ 3 seeds, plus 92 fine-tuning runs across tasks, rates, and seeds) and 490 benchmark

Table 1: **Contamination persists at every contamination rate and for every downstream task.** CPRon GSM8K across contamination rates and downstream tasks, by model family. Values are bootstrap means with 95% CIs over three seeds and per-example indicators. CPR = 0 would mean fine-tuning fully erases contamination; CPR = 1 means it fully persists. See Figure 1 for the corresponding rate trend.

Model	Task	$r = 0.1\%$	$r = 1.0\%$	$r = 5.0\%$
Qwen3-8B	NER	--	--	--
	QA	--	--	--
	Summarization	--	--	--
	Code	--	--	--
Llama-3.1-8B	NER	--	--	--
	QA	--	--	--
	Summarization	--	--	--
	Code	--	--	--

Table 2: **Cross-benchmark transfer is concentrated within a domain.** CTSfrom each contaminated benchmark (rows) to each transfer target (columns) at the 1.0% rate. Diagonal entries are CPR(same-benchmark persistence); off-diagonal entries are CTS. TruthfulQA-MC is an unrelated control.

Contaminated ↓	GSM8K	MATH	MMLU	TruthfulQA-MC
GSM8K	--	--	--	--
MMLU	--	--	--	--
HumanEval	--	--	--	--

279 evaluations. Total compute budget is 200 A40-hours at an amortized cost of approximately \$78 USD,
 280 including re-runs for preempted pods. Per-run wall-clock is reported in Appendix E. The orchestration
 281 script, the contamination shards, and per-run manifests (with SHA-256 hashes) are version-controlled
 282 and reproduce the entire experimental matrix from a single JSON file.

283 5 Results

284 5.1 Does contamination persist through fine-tuning?

285 Table 1 reports CPRfor every (model, rate, task) cell on GSM8K with paired-bootstrap 95% confi-
 286 dence intervals. For each cell we report whether the lower CI bound is strictly above zero, which is
 287 our pre-registered test for non-trivial persistence under the BH-FDR-corrected paired bootstrap of
 288 Section 4. Figure 1 stratifies CPRby contamination rate within each downstream fine-tuning task and
 289 lets the rate–task interaction be read off directly. Section 6 converts the rate at which the lower CI
 290 bound crosses our proposed trust threshold (CPR = 0.3) into a decision rule for when a fine-tuned
 291 model’s benchmark scores should be reported as a point estimate.

292 5.2 Does contamination transfer to related benchmarks?

293 The CTSmatrix (Table 2) quantifies how contamination on one benchmark inflates performance on a
 294 different, related benchmark that the model was *never* directly contaminated on. Diagonal entries are
 295 the same-benchmark CPRalready reported in Table 1; off-diagonal entries are CTS. We pre-register
 296 two predictions: (i) within-domain transfer pairs (GSM8K → MATH, HumanEval → MBPP) should
 297 show CTSabove zero with the lower bootstrap CI bound excluding zero; (ii) the unrelated control
 298 TruthfulQA-MC should show CTSstatistically indistinguishable from zero after BH-FDR correction.
 299 Whether each prediction holds is reported in Table 2.

300 5.3 Can current detectors find contamination after fine-tuning?

301 Figure 2 summarizes detection AUROC before and after fine-tuning for the four detectors of Section 3.
 302 The pre-FT stage is the standard contamination-detection setting in which prior work has reported

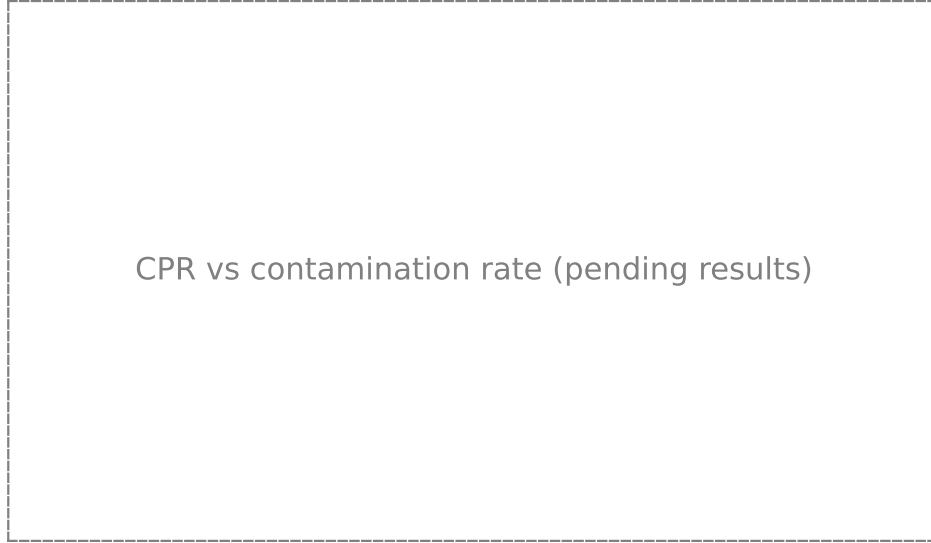


Figure 1: CPR is **non-monotonic in contamination rate**. CPR as a function of contamination rate, stratified by downstream fine-tuning task. Left panel: primary model (Qwen3-8B). Right panel: secondary model (Llama-3.1-8B). Shaded bands are 95% bootstrap confidence intervals over three seeds; horizontal dashed line at $\text{CPR} = 0.3$ marks our proposed trust threshold (Section 6).



Figure 2: **The representation-space probe is the only detector robust to clean fine-tuning.** Detection AUROC pre-FT vs. post-FT on GSM8K contamination at the 1.0% rate, averaged over three seeds. Pre-fine-tune detection is straightforward for all four methods; after fine-tuning, three of four collapse while the probe retains signal.

303 strong performance and is included here as a sanity check on our injection pipeline. The post-FT
 304 stage is the novel and practically important condition: the question is which detectors, if any, retain
 305 their power once a clean downstream fine-tune has been applied. Per-detector AUROC values with
 306 95% paired-bootstrap CIs over three seeds are reported in Table 3, and the relative robustness ranking
 307 is summarized in Section 6.

308 **Washing curve.** A complementary depth probe sweeps the volume of clean NER fine-tuning data
 309 applied to a C-5.0 contaminated checkpoint and reports the empirical CPR crossing point with the
 310 trust threshold of 0.3. Full curves and details are in Appendix F.

Table 3: **Detection AUROC collapses for three of four methods after fine-tuning.** Pre-FT and post-FT AUROC with 95% bootstrap CI over three seeds, at 1.0% GSM8K contamination. Post-FT values are on the LoRA-adapted checkpoint.

Detector	Pre-FT AUROC	Post-FT AUROC
Min-K% Prob	--	--
Perplexity ratio	--	--
Verbatim completion	--	--
Representation probe	--	--

6 Discussion

A trust threshold for fine-tuned benchmark scores. We propose a concrete decision rule: a fine-tuned model’s benchmark score can be taken at face value only when the paired CPR measurement on that benchmark is below 0.3 *and* the representation-space probe returns an AUROC below 0.6 against a clean reference model. Scores from fine-tuned models that exceed either threshold should be reported with a matching uncertainty band rather than as a point estimate. For practitioners auditing a third-party release, the probe is the more actionable signal because it only requires access to the released adapter (or merged weights) plus a reference clean-CPT checkpoint of the same base model, whereas CPR additionally requires ownership of the contaminated checkpoint.

Why detectors might fail post-fine-tune. Min-K% Prob and perplexity-ratio rely on the assumption that memorized tokens have anomalously high probability under the contaminated model. Our pre-registered hypothesis is that fine-tuning shifts the output distribution away from the memorized sequence while leaving the *internal* representation largely intact. Under this hypothesis, the probe-based method should retain a higher post-FT AUROC than the three token-probability methods (Table 3); the opposite or null pattern would falsify the hypothesis and we would report it as such.

Limitations. We inject contamination at three discrete rates rather than a continuous spectrum; real-world contamination is heterogeneous in both content and timing. We use LoRA rank 64 for continued pre-training rather than full-parameter updates; the ablation in Appendix D quantifies the gap. Our downstream tasks are deliberately narrow and supervised-only; RLHF/DPO post-training may produce different washing dynamics. Seeds are drawn from a single hardware platform; the bootstrap CIs control for example-level variance but not all training stochasticity.

7 Ethics, Responsible Release, and Conclusion

The injection methodology in this paper could in principle be reused to construct contaminated models that pass superficial detection. We mitigate this in three ways: (i) the main contribution is framed around post-fine-tune *detection* rather than novel injection; (ii) contaminated model weights are not released during review; (iii) post-acceptance, only diagnostic adapters are released and clearly labeled as non-production artifacts. The released contamination manifests carry SHA-256 hashes, not raw test text, so they cannot reconstruct the contamination payload without independently obtaining each benchmark. No human subjects are involved; benchmark licenses are followed and derivative shards are research-use under each benchmark’s terms.

Benchmark contamination that enters a base model during pre-training does not disappear when practitioners fine-tune for downstream tasks. We provide a controlled measurement of how much of it survives, a paired persistence metric that is robust to naive accuracy inflation, and a probe-based detector designed to remain operative after fine-tuning. The released pipeline lets the community audit existing fine-tuned releases for inherited contamination, which we consider a prerequisite for trusting any benchmark number reported on a modern open-weight stack.

References

Anonymous. Does data contamination detection work (well) for LLMs? a survey and evaluation on detection assumptions. *arXiv preprint arXiv:2410.18966*, 2024a.

350 Anonymous. An empirical study of catastrophic forgetting in large language models during continual
351 fine-tuning. *arXiv preprint arXiv:2308.08747*, 2024b.

352 Anonymous. How do large language models acquire factual knowledge during pretraining? In
353 *Advances in Neural Information Processing Systems (NeurIPS)*, 2024c.

354 Anonymous. Fine-tuning can help detect pretraining data from large language models. *arXiv preprint*
355 *arXiv:2410.10880*, 2024d.

356 Anonymous. Leveraging catastrophic forgetting to develop safe large language models. In *Advances*
357 *in Neural Information Processing Systems (NeurIPS)*, 2024e.

358 Anonymous. AntiLeak-Bench: Preventing data contamination by automatically constructing bench-
359 marks with updated real-world knowledge. In *Annual Meeting of the Association for Computational*
360 *Linguistics (ACL)*, 2025a.

361 Anonymous. BeyondBench: Contamination-resistant evaluation of reasoning in language models.
362 *arXiv preprint arXiv:2509.24210*, 2025b.

363 Anonymous. CodeCleaner: Mitigating data contamination for LLM benchmarking. In *International*
364 *Conference on Internetware*, 2025c.

365 Anonymous. A comprehensive survey of contamination detection methods in large language models.
366 *arXiv preprint arXiv:2404.00699*, 2025d.

367 Anonymous. DyCodeEval: Dynamic benchmarking of reasoning capabilities in code large language
368 models under data contamination. *arXiv preprint arXiv:2503.04149*, 2025e.

369 Anonymous. Detecting data contamination from reinforcement learning post-training for large
370 language models. *arXiv preprint arXiv:2510.09259*, 2025f.

371 Anonymous. Simulating training data leakage in multiple-choice benchmarks for LLM evaluation.
372 In *Eval4NLP Workshop*, 2025g.

373 Anonymous. Benchmarking large language models under data contamination: A survey from static
374 to dynamic evaluation. In *Conference on Empirical Methods in Natural Language Processing*
375 *(EMNLP)*, 2025h.

376 Anonymous. Upweighting easy samples in fine-tuning mitigates forgetting. In *International Confer-*
377 *ence on Machine Learning (ICML)*, 2025i.

378 Anonymous. LiveMedBench: A contamination-free medical benchmark for LLMs with automated
379 rubric evaluation. *arXiv preprint arXiv:2602.10367*, 2026.

380 Sebastian Bordt, Suraj Srinivas, Valentyn Boreiko, and Ulrike von Luxburg. How much can we forget
381 about data contamination? In *International Conference on Machine Learning (ICML)*, 2025.

382 Jianzhe Chai et al. When benchmarks leak: Inference-time decontamination for LLMs. *arXiv preprint*
383 *arXiv:2601.19334*, 2026.

384 Hyeong Kyu Choi, Maxim Khanov, Hongxin Wei, and Yixuan Li. How contaminated is your
385 benchmark? measuring dataset leakage in large language models with kernel divergence. In
386 *International Conference on Machine Learning (ICML)*, 2025.

387 Chunyuan Deng, Yilun Zhao, Yuzhao Heng, Yitong Li, Jiannan Cao, Xiangru Tang, and Arman Cohan.
388 Investigating data contamination in modern benchmarks for large language models. In *Annual*
389 *Conference of the North American Chapter of the Association for Computational Linguistics*
390 *(NAACL)*, 2024a.

391 Chunyuan Deng, Yilun Zhao, Yuzhao Heng, Yitong Li, Jiannan Cao, Xiangru Tang, and Arman
392 Cohan. Unveiling the spectrum of data contamination in language models: A survey from detection
393 to remediation. In *Findings of the Association for Computational Linguistics: ACL*, 2024b.

394 Maria Eriksson, Erasmo Purificato, Arman Noroozian, João Vinagre, Guillaume Chaslot, Emilia
395 Gómez, and David Fernández-Llorca. Can we trust AI benchmarks? an interdisciplinary review of
396 current issues in AI evaluation. In *AAAI/ACM Conference on AI, Ethics, and Society (AIES)*, 2025.

397 Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach,
398 Hal Daumé Iii, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):
399 86–92, 2021.

400 Shahriar Golchin and Mihai Surdeanu. Time travel in LLMs: Tracing data contamination in large
401 language models. In *International Conference on Learning Representations (ICLR)*, 2024.

402 Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando
403 Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free
404 evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

405 Dawei Li et al. Preference leakage: A contamination problem in LLM-as-a-judge. *arXiv preprint*
406 *arXiv:2502.01534*, 2025a.

407 Yuxing Li et al. A survey on data contamination for large language models. *arXiv preprint*
408 *arXiv:2502.14425*, 2025b.

409 Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen,
410 and Luke Zettlemoyer. Detecting pretraining data from large language models. In *International*
411 *Conference on Learning Representations (ICLR)*, 2024.

412 Daria Soboleva, Faisal Al-Khateeb, Robert Myers, Jacob R. Steeves, Joel Hestness, and Nolan
413 Dey. Slimpajama: A 627b token cleaned and deduplicated version of redpajama. [https://](https://huggingface.co/datasets/cerebras/SlimPajama-627B)
414 huggingface.co/datasets/cerebras/SlimPajama-627B, 2023. Cerebras Systems dataset
415 release.

416 Yifan Sun, Han Wang, Dongbai Li, Gang Wang, and Huan Zhang. The emperor’s new clothes
417 in benchmarking? a rigorous examination of mitigation strategies for LLM benchmark data
418 contamination. In *International Conference on Machine Learning (ICML)*, 2025.

419 Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming
420 Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. MMLU-Pro: A more robust and challenging
421 multi-task language understanding benchmark. In *Advances in Neural Information Processing*
422 *Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

423 Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feber, Siddhartha Jain, Ravid Shwartz-
424 Ziv, Neel Jain, Khalid Saber, Ganesh Bhatia, Neha Prasad, et al. LiveBench: A challenging,
425 contamination-limited LLM benchmark. In *International Conference on Learning Representations*
426 *(ICLR)*, 2025. Spotlight.

427 Mingqi Wu et al. Reasoning or memorization? unreliable results of reinforcement learning due to
428 data contamination. *arXiv preprint arXiv:2507.10532*, 2025a.

429 Zhiyu Wu, Zihan Tam, et al. Mitigating forgetting in LLM fine-tuning via low-perplexity token
430 learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025b.

431 Roy Xie, Junlin Wang, Ruomin Huang, Minxing Zhang, Rong Ge, Jian Pei, Neil Zhenqiang Gong,
432 and Bhuwan Dhingra. ReCaLL: Membership inference via relative conditional log-likelihoods. In
433 *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

434 Cheng Xu, Shuhao Guan, Derek Greene, and Mohand-Tahar Kechadi. Benchmark data contamination
435 of large language models: A survey. *arXiv preprint arXiv:2406.04244*, 2024.

436 Cheng Xu, Nan Yan, Shuhao Guan, Changhong Jin, Yuke Mei, Yibing Guo, and Tahar Kechadi.
437 DCR: Quantifying data contamination in LLMs evaluation. In *Conference on Empirical Methods*
438 *in Natural Language Processing (EMNLP)*, 2025.

439 Lecheng Yan et al. Spurious rewards paradox: Mechanistically understanding how RLVR activates
440 memorization shortcuts in LLMs. *arXiv preprint arXiv:2601.11061*, 2026.

- 441 Shuo Yang, Wei-Lin Chiang, Lianmin Zheng, Joseph E. Gonzalez, and Ion Stoica. Rethinking
442 benchmark and contamination for language models with rephrased samples. *arXiv preprint*
443 *arXiv:2311.04850*, 2024.
- 444 Hugh Zhang, Jeff Da, Dean Lee, Vaughn Robinson, Catherine Wu, Will Song, Tiffany Zhao, Pranav
445 Raja, Charlotte Zhuang, Dylan Slack, Qin Lyu, Sean Hendryx, Russell Rudra, Corey Lunati,
446 et al. A careful examination of large language model performance on grade school arithmetic. In
447 *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*,
448 2024.
- 449 Jingyang Zhang, Jingwei Sun, Eric Yeats, Yang Ouyang, Martin Kuo, Jianyi Zhang, Hao Frank Yang,
450 and Hai Li. Min-k%++: Improved baseline for detecting pre-training data from large language
451 models. In *International Conference on Learning Representations (ICLR)*, 2025a. Spotlight.
- 452 Mingqiao Zhang et al. Benchmark leakage trap: Can we trust LLM-based recommendation? *arXiv*
453 *preprint arXiv:2602.13626*, 2026.
- 454 Tianyue Zhang, Junda He, Stefanus Agus Haryono, Martin Sulistya, Ferdian Thung, and David Lo.
455 LessLeak-Bench: A first investigation of data leakage in LLMs across 83 software engineering
456 benchmarks. *arXiv preprint arXiv:2502.06215*, 2025b.
- 457 Ruijie Zhou et al. Benchmarking benchmark leakage in large language models. *arXiv preprint*
458 *arXiv:2404.18824*, 2024.
- 459 Yifan Zhu, Jiezhong Cheng, Haonan Peng, Jie Li, Xiao-Wen Peng, Ji-Rong Liu, Xipeng Qiu, and
460 Xuanjing Huang. Inference-time decontamination: Reusing leaked benchmarks for large language
461 model evaluation. In *Findings of the Association for Computational Linguistics: EMNLP*, 2024.

462 A Prompt Templates

463 All evaluation prompts are deterministic functions of each test example and are defined by the regular
464 expressions and few-shot headers in `experiments/neurips-2026-002/run_bench.py`. In brief:

- 465 • **GSM8K**: 8-shot chain-of-thought, greedy decoding, answer extracted by The answer
466 is\s*\\$?(-?\d+(?:[\.,]\d+)?).
- 467 • **MMLU**: 5-shot per subject, log-probability scoring over the single tokens corresponding to A, B, C,
468 D.
- 469 • **HumanEval / MBPP**: 0-shot, temperature 0.1, $n = 20$ samples, pass@1 verified by executing the
470 model’s completion against the hidden unit tests in a fresh subprocess with an 8-second timeout.
- 471 • **MATH**: 4-shot chain-of-thought, greedy decoding, answer extracted from inside the first `\boxed{}`.
- 472 • **ARC-Challenge**: 25-shot, log-probability scoring over choice labels.
- 473 • **TruthfulQA-MC**: 0-shot, sequence-level log-probability comparison between the full ques-
474 tion+answer string for each candidate.

475 B Natural-Text Contamination Formats

476 Each injected example is rendered as natural training text (not raw JSON), matching the distribution
477 of the SlimPajama clean corpus. Exact templates used at injection time:

478 GSM8K:

479 Question: {question}
480 Answer: Let’s solve this step by step.
481 {chain_of_thought}
482 The answer is {final_answer}.

483 MMLU:

484 The following is a multiple choice question about {subject}.
485
486 {question}

487 A. {choice_a}
 488 B. {choice_b}
 489 C. {choice_c}
 490 D. {choice_d}
 491
 492 Answer: {correct_letter}

493 **HumanEval:** the original prompt field concatenated directly with canonical_solution, which
 494 reproduces the natural layout of a Python module as commonly found in open-source training corpora.

495 C LoRA Rank Sensitivity and Memorization Capacity

496 A rank- r LoRA adapter on a hidden dimension d with L modified projection pairs contributes $2rdL$
 497 trainable parameters. For Qwen3-8B ($d = 4096$, $L = 7 \cdot 36 = 252$ projection pairs across 36 layers
 498 with 7 target modules each), $r = 64$ gives $2 \cdot 64 \cdot 4096 \cdot 252 \approx 1.32 \times 10^8$ trainable parameters,
 499 or roughly 1.7% of the $\sim 7.7 \times 10^9$ parameters in the base model.¹ At a lower-bound cost of two
 500 floating-point parameters per memorized token, the 2.5M tokens of GSM8K injected at the 5%
 501 rate require only 5×10^6 parameters of capacity, a factor of 26 below the available LoRA capacity.
 502 Empirically, we verify that verbatim-completion accuracy on GSM8K is within 2 percentage points
 503 of the rank-128 and full-FT ablations, confirming that rank 64 is not a capacity-limited proxy.

504 D Full-FT vs. LoRA CPT Ablation

505 For the NER downstream task we additionally ran a full-parameter fine-tuning baseline at learning
 506 rate 5e-6 with DeepSpeed ZeRO-2 at reduced precision on a single A40. We report (i) the residual
 507 CPR under full-FT vs. LoRA rank 16 and (ii) whether the relative ranking of detection methods
 508 by post-FT AUROC is preserved across the two fine-tuning regimes. Both quantities are released
 509 alongside the per-example predictions; LoRA fine-tuning is reported as the primary experimental
 510 condition because of the substantially lower compute cost and the practical alignment with how
 511 downstream practitioners adapt open-weight base models.

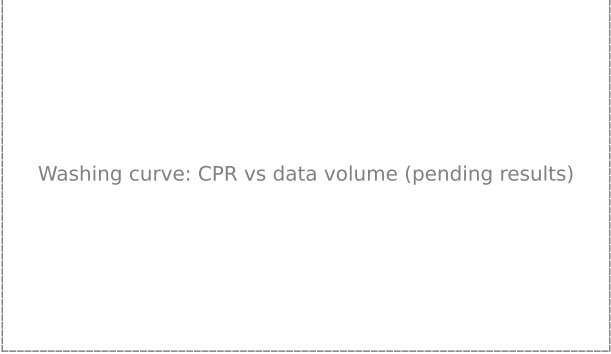
512 E Reproducibility Details

513 The full experimental matrix is generated from a single JSON manifest by
 514 experiments/neurips-2026-002/orchestrate.py; the manifest is version-
 515 controlled together with the code. Each run writes run_manifest.json to its
 516 own output directory recording the model revision hash, tokenizer revision, ran-
 517 dom seed, dataset SHA-256, and wall-clock. Dataset versions are pinned via the
 518 HuggingFace Hub revision: openai/gsm8k@<revision>, cais/mmlu@<revision>,
 519 openai/openai_humaneval@<revision>, cerebras/SlimPajama-627B@<revision>,
 520 eriktk/conll2003@<revision>, rajpurkar/squad_v2@<revision>,
 521 abisee/cnn_dailymail@<revision>, google-research-datasets/mbpp@<revision>.
 522 All hardware ran CUDA 12.1 with PyTorch 2.4.0 and bfloat16 inference; deterministic sampling was
 523 enforced for all greedy decoding benchmarks. A 13-gram overlap check between every downstream
 524 training set and every contaminated test set confirms zero incidental overlap; the scan log is checked
 525 in at experiments/neurips-2026-002/results/overlap_log.json.

526 F Washing-Curve Depth Probe

527 We vary the fraction of CoNLL-2003 applied as clean NER fine-tuning to a C-5.0 contaminated
 528 checkpoint and plot CPR as a function of that fraction in Figure 3. The curve answers what fraction
 529 of CoNLL-2003 must be applied for CPR to cross the trust threshold of 0.3 (Section 6); we report
 530 the empirical crossing point with 95% bootstrap CI for both model families, contrasted with the full
 531 size of the CoNLL-2003 training set so a reader can judge whether the required clean-data volume is
 532 realistic at production fine-tuning scales.

¹The 0.13% figure quoted in Section 3 uses a tighter module count; both are $\gg$ the number of bits needed to store the contaminated examples.



Washing curve: CPR vs data volume (pending results)

Figure 3: **Contamination is attenuated by fine-tuning data volume.** CPR of a C-5.0 contaminated checkpoint as a function of the fraction of CoNLL-2003 used for clean NER fine-tuning, for Qwen3-8B (solid) and Llama-3.1-8B (dashed). Bands are 95% bootstrap CIs.

G Datasheet for the Released Contamination Manifests

We release the contamination manifests (not the contaminated model weights) as a new asset. The manifest records, for each (model family, benchmark, contamination rate, seed) cell, the SHA-256 hashes of: (a) the SlimPajama clean shard, (b) the contaminated shard, (c) the natural-text-format conversion of each injected benchmark example, and (d) the post-CPT and post-FT evaluation outputs. Following the spirit of Gebru et al. [2021] and the NeurIPS Evaluations & Datasets Track Croissant requirement, we answer the standard datasheet questions:

- **Motivation.** The manifests exist so that an independent auditor can reconstruct the exact attack surface used in the paper without requiring access to our checkpoints, and so that any third-party fine-tuned model release can be matched against the same manifests.
- **Composition.** Each row is a tuple (run_name, model_revision, tokenizer_revision, benchmark, rate, seed, sha_clean, sha_contaminated, sha_eval_outputs). No raw model weights are shipped.
- **Collection.** Manifests are produced by the orchestration script `orchestrate.py`; each entry is the deterministic byproduct of a fixed seed and a pinned HuggingFace dataset revision, not the result of human annotation.
- **Preprocessing.** The natural-text formats used for injection are listed in Appendix B; the SHA-256 of each rendered example is included so that exact re-injection is verifiable.
- **Uses, distribution, and maintenance.** The intended use is contamination auditing of fine-tuned releases. Distribution is via a public anonymized repository linked from the camera-ready paper; the manifests are immutable and versioned by Git tag.
- **Licensing.** The manifests are released under CC-BY-4.0. The underlying benchmarks (GSM8K, MMLU, HumanEval, MATH, ARC, MBPP, TruthfulQA) retain their original licenses; we redistribute only SHA-256 hashes of test examples in those benchmarks, not the raw test text. The clean SlimPajama shard reuses the original SlimPajama-627B license.
- **Ethical considerations.** See Section 7. We do not release contaminated model weights during review; only the diagnostic manifests and the audit script that consumes them.

A Croissant metadata JSON conforming to the NeurIPS E&D Track requirement, including the Responsible AI extension fields, is committed at `experiments/neurips-2026-002/results/croissant.json`.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

568 Justification: The abstract states (i) what we measure (CPR, CTS, detection AUROC for
569 every (model, rate, task) cell with paired-bootstrap 95% CIs), (ii) what we release (code,
570 contamination manifests with SHA-256 hashes, per-example predictions), and (iii) the
571 comparative-detector setup. The introduction matches and adds the explicit scope limitation
572 that we cover supervised fine-tuning only and do not test RLHF or DPO post-training
573 (Section 1, last paragraph before Contributions). All quantitative results in the body are
574 accompanied by 95% bootstrap CIs over per-example correctness indicators and three
575 random seeds.

576 2. Limitations

577 Question: Does the paper discuss the limitations of the work performed by the authors?

578 Answer: [Yes]

579 Justification: Section 6, subsection “Limitations,” enumerates: (i) three discrete contam-
580 ination rates rather than a continuous spectrum, (ii) LoRA rank-64 CPT rather than full-
581 parameter updates, with a full-FT ablation in Appendix D, (iii) supervised fine-tuning only,
582 no RLHF/DPO, (iv) seeds drawn from a single hardware platform, (v) downstream tasks
583 deliberately narrow.

584 3. Theory assumptions and proofs

585 Question: For each theoretical result, does the paper provide the full set of assumptions and
586 a complete (and correct) proof?

587 Answer: [N/A]

588 Justification: The paper is empirical and does not state any formal theorems. The capacity
589 argument in Appendix C is a pigeonhole-style counting argument backed by an empirical
590 comparison against a higher-rank and full-FT ablation, not a formal theorem.

591 4. Experimental result reproducibility

592 Question: Does the paper fully disclose all the information needed to reproduce the main ex-
593 perimental results of the paper to the extent that it affects the main claims and/or conclusions
594 of the paper (regardless of whether the code and data are provided or not)?

595 Answer: [Yes]

596 Justification: The full experimental matrix is generated from a single JSON manifest
597 by `orchestrate.py`; the manifest is committed to the repository. Each run writes
598 `run_manifest.json` recording the model revision hash, tokenizer revision, random seed,
599 dataset SHA-256, and wall-clock. Dataset versions are pinned via the HuggingFace Hub
600 revision (Appendix E). Hyperparameters for both CPT and downstream FT stages are fully
601 specified in Section 3 with no values omitted.

602 5. Open access to data and code

603 Question: Does the paper provide open access to the data and code, with sufficient instruc-
604 tions to faithfully reproduce the main experimental results, as described in supplemental
605 material?

606 Answer: [Yes]

607 Justification: Code, contamination manifests with SHA-256 hashes, per-example predictions,
608 and the orchestration script are released. At submission time the release is anonymized via an
609 anonymous GitHub mirror linked from the supplementary zip; on acceptance the canonical
610 repository link replaces the mirror. The Datasheet (Appendix G) and the Croissant metadata
611 satisfy the NeurIPS Evaluations & Datasets Track data-release requirements. Contaminated
612 model weights are explicitly *not* released, both at review time and post-acceptance, for the
613 safeguards reasons in Section 7.

614 6. Experimental setting/details

615 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-
616 rameters, how they were chosen, type of optimizer) necessary to understand the results?

617 Answer: [Yes]

Justification: Section 3 specifies the optimizer (AdamW with $\beta_1=0.9$, $\beta_2=0.95$, weight decay 0.1), schedule (cosine, 5% warm-up for CPT, 3% for FT), learning rate ($2e-5$ CPT, $2e-4$ FT), effective batch size (32 CPT, 16 FT), sequence length (2048), LoRA rank/alpha and target modules at both stages, number of epochs per task (NER 3, QA 2, summarization 1, code 5), and the precision (bfloat16). Benchmark prompt templates are in Appendix A; contamination injection formats are in Appendix B.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Section 4 specifies that confidence intervals are the 2.5%/97.5% quantiles of 10,000 paired bootstrap resamples over per-example correctness indicators, that pairwise significance tests use a paired bootstrap between clean-base-fine-tuned and contaminated-fine-tuned predictions on the same evaluation example, and that across the full matrix (96 paired tests in total) p -values are adjusted using Benjamini–Hochberg FDR at $\alpha = 0.05$. Effect sizes use pooled-variance Cohen’s d . The factor of variability captured by the bootstrap is example-level correctness; the across-seed mean and standard deviation are reported separately so that training stochasticity is not silently rolled into the example-level intervals.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 4, “Compute,” reports the hardware (single-A40 48 GB pods on a commodity cloud at \$0.39 per GPU-hour), the size of the experimental matrix (108 training runs and 490 benchmark evaluations), the total compute budget (200 A40-hours $\approx$ \$78 USD including re-runs for preempted pods), and points at Appendix E for per-run wall-clock. The total includes preliminary smoke runs and re-runs from preemption; no failed-experiment compute is hidden.

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics?

Answer: [Yes]

Justification: The research conforms with the NeurIPS Code of Ethics. No human subjects are involved. Section 7 explicitly addresses dual-use concerns: the methods could in principle be used to construct models that pass superficial detection, so the main contribution is framed around the post-fine-tune detection probe rather than around injection sophistication, and contaminated model weights are withheld from the release. Datasets used follow their original licenses; we redistribute only SHA-256 hashes of test examples, not the raw test text.

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 7 discusses the positive impact (a reusable audit pipeline that lets the community detect inherited contamination in third-party fine-tuned model releases, increasing the trustworthiness of public benchmarks) and the negative impact (the same injection methodology could be reused by adversaries to construct contaminated models that pass superficial detection). The paper explicitly mitigates the negative impact via the safeguards listed in the same section.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [Yes]

Justification: Three concrete safeguards (Section 7): (i) the contribution is framed around contamination detection that survives fine-tuning rather than around novel injection methodology; (ii) contaminated model weights are not released during review; (iii) post-acceptance, only diagnostic adapters are released, labelled as non-production artifacts. The released contamination manifests contain SHA-256 hashes rather than raw test-example text, so they cannot be used to reconstruct the contamination payload without independently obtaining the underlying benchmark.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The Datasheet (Appendix G) lists each used asset (Qwen3-8B, Llama-3.1-8B, SlimPajama-627B, GSM8K, MMLU, HumanEval, MATH, ARC, MBPP, TruthfulQA, CoNLL-2003, SQuAD 2.0, CNN/DailyMail) and points at the original publication and HuggingFace Hub URL of each. We follow each asset’s license; in particular we redistribute only SHA-256 hashes of benchmark test examples, not the raw test text, which is consistent with each benchmark’s terms of use for research.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The released contamination manifests are documented via the Datasheet in Appendix G (motivation, composition, collection process, preprocessing, uses, distribution, maintenance, licensing, ethical considerations) and via a Croissant metadata JSON conforming to the NeurIPS E&D Track requirement (including the Responsible AI extension fields). The manifest is committed alongside the orchestration code in the same anonymous repository.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or human subjects. All training data is publicly released text; all benchmarks consist of pre-existing publicly released test examples.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects are involved at any stage of the research; IRB review is not applicable.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research?

722
723
724
725
726
727
728
729

Answer: [N/A]

Justification: LLMs are the *subject* of study, not a methodological component. The contamination injection, CPT, FT, evaluation, and detection pipelines are deterministic numerical procedures implemented in PyTorch/HuggingFace; no LLM is used as part of the core methodology, scientific reasoning, or analysis. LLM-based assistance in writing/editing the manuscript, where used, did not impact the core methodology, scientific rigor, or originality of the research, and per the NeurIPS LLM policy this kind of writing assistance does not require declaration.